Monoid & Semigroup

Monoid · Fold invoice line totals / log strings · Build with Waqar

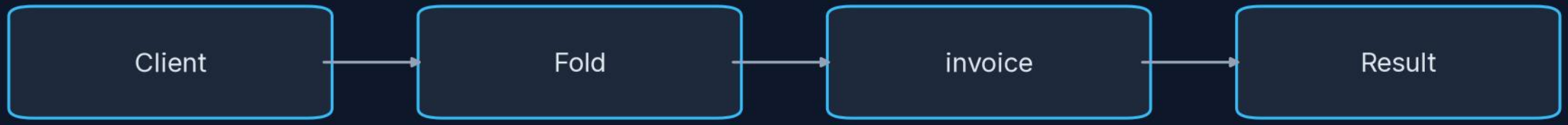
Today's Hook

- Problem: Fold invoice line totals / log strings
- Pattern: Monoid — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Fold invoice line totals / log strings
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Monoid: Fold invoice line totals / log strings

Real-World Scenario

- Running example: Fold invoice line totals / log strings
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.functional.monoid.problem;

import java.util.*;

public final class NaiveSum {
  public static int sum(List<Integer> xs){ int s=0; for(int x:xs)s+=x; return s; }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```
package com.buildwithwaqar.patterns.functional.monoid.problem;

import java.util.*;
import java.util.function.*;

interface Monoid<A> { A empty(); A combine(A a, A b); }
final class Monoids {
  static final Monoid<Integer> INT_ADD = new Monoid<>(){ public Integer empty(){return 0;} public Integer combine(Integer a,Integer b){return a+b;} };
  static final Monoid<String> STRING = new Monoid<>(){ public String empty(){return "";} public String combine(String a,String b){return a+b;} };
  static <A> A fold(Monoid<A> m, List<A> xs){ A acc=m.empty(); for(A x: xs) acc=m.combine(acc,x); return acc; }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.monoid.problem;
import java.util.*; import java.util.function.*;
public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Monoid: fold invoice totals ===");
    System.out.println("--- naive ---"); System.out.println(NaiveSum.sum(List.of(10,20,30)));
System.out.println("--- monoid ---");
System.out.println(Monoids.fold(Monoids.INT_ADD, List.of(10,20,30)));
System.out.println(Monoids.fold(Monoids.STRING, List.of("a","b","c")));
  }
}
```

Walkthrough Output

```
=== Monoid: fold invoice totals ===
--- naive ---
60
--- monoid ---
60
abc
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Monoid.

Related Patterns

- Contrast Monoid with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Monoid → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.